

# High Dimensional Problems

Steve Avsec

Illinois Institute of Technology

April 24, 2025

# Overview

## 1 When $d \gg N$

# Introduction

In this course, we have generally assumed that the number of samples is much larger than the number of features.

# Introduction

In this course, we have generally assumed that the number of samples is much larger than the number of features.

There are numerous real-world problems where this is not the case:

- 1 Gene classification
- 2 Malware classification
- 3 Text-based data

# Introduction

In this course, we have generally assumed that the number of samples is much larger than the number of features.

There are numerous real-world problems where this is not the case:

- 1 Gene classification
- 2 Malware classification
- 3 Text-based data

Many of the techniques we've used are not valid (think back to the beginning of the course and the "matrix pictures" we used.)

# A Dimensionality Trick

Let  $X$  be the data matrix ( $N$  by  $d$ ).

# A Dimensionality Trick

Let  $X$  be the data matrix ( $N$  by  $d$ ).

Let

$$X = U\Sigma V^T$$

be the singular value decomposition.

# A Dimensionality Trick

Let  $X$  be the data matrix ( $N$  by  $d$ ).

Let

$$X = U\Sigma V^T$$

be the singular value decomposition.

In this case  $U$  is  $N \times N$  (orthogonal matrix),  $D$  is diagonal with decreasing values down the diagonal (also  $N \times N$ ), and  $V$  is  $p \times D$  with orthonormal columns.



# A Dimensionality Trick

Let  $X$  be the data matrix ( $N$  by  $d$ ).

Let

$$X = U\Sigma V^T$$

be the singular value decomposition.

In this case  $U$  is  $N \times N$  (orthogonal matrix),  $D$  is diagonal with decreasing values down the diagonal (also  $N \times N$ ), and  $V$  is  $p \times D$  with orthonormal columns.

Let  $R = UD$ . This is an  $N \times N$  matrix which "captures" all of the information in the full data matrix.

# Kernels

Suppose we have a set of documents. Let

$$\Phi_m(x) = \phi_a(x)_{a \in A_m}$$

where  $m$  is a fixed number and  $A_m$  is the set of substrings of length  $m$ , and  $\phi_a(x)$  is the number of times  $a$  occurs in  $x$ .

# Kernels

Suppose we have a set of documents. Let

$$\Phi_m(x) = \phi_a(x)_{a \in A_m}$$

where  $m$  is a fixed number and  $A_m$  is the set of substrings of length  $m$ , and  $\phi_a(x)$  is the number of times  $a$  occurs in  $x$ .

Let

$$K_m(x_1, x_2) = \langle \Phi_m(x_1), \Phi_m(x_2) \rangle$$

.

# Kernels

Suppose we have a set of documents. Let

$$\Phi_m(x) = \phi_a(x)_{a \in A_m}$$

where  $m$  is a fixed number and  $A_m$  is the set of substrings of length  $m$ , and  $\phi_a(x)$  is the number of times  $a$  occurs in  $x$ .

Let

$$K_m(x_1, x_2) = \langle \Phi_m(x_1), \Phi_m(x_2) \rangle$$

.

This kernel is computable using some tree-based data structure tricks without computing the actual vectors.

# Kernels

Suppose we have a set of documents. Let

$$\Phi_m(x) = \phi_a(x)_{a \in A_m}$$

where  $m$  is a fixed number and  $A_m$  is the set of substrings of length  $m$ , and  $\phi_a(x)$  is the number of times  $a$  occurs in  $x$ .

Let

$$K_m(x_1, x_2) = \langle \Phi_m(x_1), \Phi_m(x_2) \rangle$$

.

This kernel is computable using some tree-based data structure tricks without computing the actual vectors.

We can use this kernel matrix ( $N \times N$ ) to train a classifier *without* dealing with the high-dimensional nature of this problem.